

Rapport intermédiaire pour le “Projet de programmation”

Pour la date du 15 Mars, 2025

Cotrez Jules et Shay Ronen

15 mars 2025

1 Introduction

1.1 But du projet

Le but de notre projet est d’implémenter dans un langage fonctionnel (ici Ocaml) un jeu de plateformes en deux dimensions, dans lequel l’objectif est de réaliser des niveaux consécutifs en combattant des ennemis ainsi qu’en s’aidant d’items dispersés un peu partout dans le jeu.

1.2 Métrique

Nous considérerons notre projet comme réussi si nous parvenons à réaliser un jeu complet avec des déplacements (au minimum les déplacements classiques , un grappin qui permet un mouvement de balancier, et un jetpack), plusieurs niveaux, des ennemis et des items divers. Le reste (jeu en réseau, IA pour les ennemis, animation pour les attaques, ...) sera considéré comme du bonus.

2 Implémentation

2.1 réalisation du projet

La réalisation du projet suis plusieurs étape pour chaque ajout d’élément. D’abord nous ajoutons l’élément à la structure du jeu puis nous l’affichons si il y a besoin de l’afficher et enfin nous implementons les relations entre l’élément ajouté et les éléments existant. Par exemple pour l’ajout des plateforme nous ajoutons la structure et les attributs associés ensuite nous l’affichons et enfin nous implementons les colisions avec le personnage pour qu’il puisse se tenir dessus. Nous réalisons le projet en nous attribuant chacun une des tâches du du planning lié à un élément.

2.2 partie du logiciel codée

Pour le moment nous avons codé les déplacements du joueur (grappin inclus), l’affichage d’un menu ainsi que du premier niveau (avec plateformes et un ennemi), le parsing de fichiers Json afin de facilement modifier la description des niveaux (nombre et position des plateformes et des ennemis). De plus nous avons veillé à implémenter la plus grande partie possible de manière fonctionnelle en évitant les effets de bord (c’est là toute la difficulté et l’intérêt de notre projet !). Les uniques effets de bords proviennent de l’utilisation de librairie Raylib pour l’affichage de l’interface graphique.

2.3 structure en modules/packages

Nous avons utilisé le gestionnaire Dune pour instancier la structure de notre projet. Actuellement, nous avons deux classes principales séparées dans deux fichiers ml (+ deux fichiers mli associés pour l’interface) : jeu et joueur. La classe joueur contient tout ce qui est nécessaire à l’instanciation d’un nouveau joueur (joueur principal comme ennemi, nous ferons la distinctions dans l’implémentation du jeu). Elle possède donc tous les attributs utiles tels que les dimensions, la direction, le vecteur de position, ... La classe jeu contient l’initialisation et la boucle principale du jeu. Elle possède également des structures facilitant la gestion des différentes entités (joueurs, ennemis, plateformes).

2.4 représentation des données

Le projet étant un jeu nous avons donc besoin de stocker les données du terrain, du joueur, des ennemis, et des objets récupérable. Etant donné que nous le faisons en fonctionnel (ocaml) nous n’utilisons pas d’objet pour enregistrer ses données. A la place nous utilisons des structures qui sont appelés dans la fonction de boucle du jeu et que nous modifions au cours de l’exécution. La structure entites (4) regroupe toutes les types d’entités utiliser : le personnage du joueur (player), la liste des ennemi (ennemi list), la liste des plateformes (plateforme list). Le jeu possède plusieurs niveau avec des emplacements de plateformes, d’ennemi et d’objets différents que nous stockons dans plusieurs fichiers .json, ainsi nous créons un nouveau niveau en ajoutant un fichier levelX.json (X étant le numéro du niveau).

2.5 technologies

- OCaml : Langage de programmation fonctionnelle utilisé pour le développement du jeu
- Raylib : Bibliothèque d’OCaml permettant l’affichage graphique du jeu
- JSON : Format de fichier utilisé pour la définition des niveaux
- Dune : Système de construction permettant la simplification de la compilation et la gestion de la structure du projet

3 Jalons

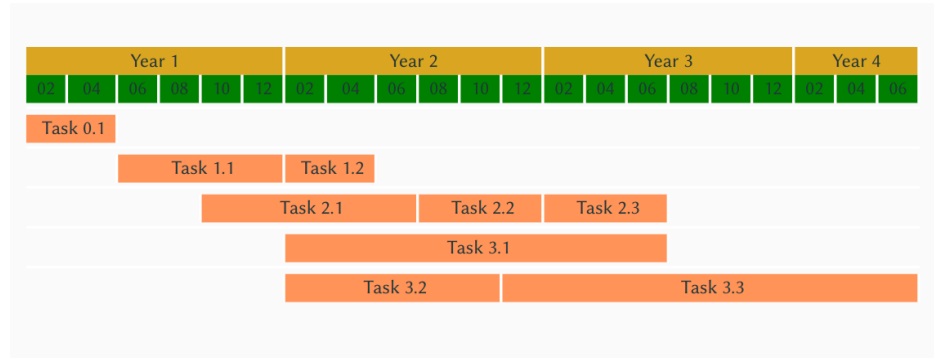


FIGURE 1 – Planning des taches du projet

3.1 tâche restante

Nous avons encore à ajouter des ennemis, les objets collectables, le déclencheur déterminant la fin du niveau et le passage au suivant (le personnage doit entrer dans une porte pour passer au niveau suivant), et le système de combat permettant d'éliminer les ennemis (une attaque en mêlée avec une épée et une attaque à distance en lançant un projectile). En bonus nous pouvons faire l'ajout du jeu en réseaux permettant à deux personnes de contrôler chacun un personnage et l'ajout d'une intelligence artificiel pour les ennemis leur permettant de se déplacer de manière optimal pour gêner le joueur.

3.2 organisation des taches

Le calendrier (3) ci dessus montre la répartition et la date limite de chaque tâche. Pour mener le projet à sa conclusions nous nous concentrons à présent sur l'implementation des derniers aspect cruciaux du jeu incluant les ennemis, les combats (attaque du personnage) et la fin de chaque niveaux et passage au suivant.

3.3 taches terminées

Nous avons fini d'implémenter la physique des déplacements du joueur incluant les déplacements simple (gauche, droite et saut) ainsi que la mécanique de pendule lié à l'utilisation du grapin. La plateforme sont implémenté avec la possibilité du personnage de se maintenir dessus. L'affichage du personnage, des plateformes et des ennemis est terminées.

4 Annexe

```
type plateforme = {  
  platform_x : int;  
  platform_y : int;  
  platform_width : int;  
  platform_height : int;  
}  
  
type entities = {  
  player : joueur;  
  ennemis : ennemi list;  
  plateforme_list : plateforme list;  
}  
  
type joueur = {  
  nom : string;  
  pos : position;  
  vector_velocity : float * float;  
  health_point : int;  
  attack_point : int;  
  jetpack_carburant_pourcentage : int;  
  grap : grappin;  
  sprite : string;  
  height : float;  
  width : float;  
  facing_right : bool;  
  falling : bool  
}
```

FIGURE 2 – structure de données des élément du jeu
